

Build Automation & Dependency Management

3 hands-on labs: Maven lifecycle, Gradle basics, and dependency conflict resolution

LABS IN THIS GUIDE

- Lab 1 — Build and run a Maven project through its full lifecycle
- Lab 2 — Build the same project in Gradle and compare behavior
- Lab 3 — Create and resolve a transitive dependency conflict

Student lab guide · Companion to "Build Automation & Dependency Management" (Part 1 of 3)

1 Lab 1: Maven Build Lifecycle

Objective: Create a Maven project from scratch, run it through every lifecycle phase, and observe what each phase actually does.

Prerequisites

- JDK 11+ installed (`java -version`)
- Maven installed (`mvn -version`)

Steps

Step 1 — Generate a new Maven project

```
mvn archetype:generate -DgroupId=com.training.demo \
-DartifactId=maven-lifecycle-lab \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DarchetypeVersion=1.4 -DinteractiveMode=false
cd maven-lifecycle-lab
```

Step 2 — Inspect the generated pom.xml

Open `pom.xml` and identify: `groupId`, `artifactId`, `version`, and the default `junit` dependency already declared. Note the GAV coordinates — these are exactly what was covered in the source document under "Coordinates."

Step 3 — Run each lifecycle phase individually

Run these one at a time and observe the console output before moving to the next:

```
mvn validate
mvn compile
mvn test
mvn package
mvn verify
mvn install
```

After `mvn package`, confirm a JAR file now exists:

```
ls target/*.jar
```

Step 4 — Add a real dependency

Add Jackson to `pom.xml`:

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.15.0</version>
</dependency>
```

Re-run `mvn compile` and confirm Maven downloads it into `~/.m2/repository`:

```
find ~/.m2/repository -iname "jackson-databind*"
```

Verification Checklist

- [] JAR file exists under `target/` after `package`
- [] Jackson dependency appears in local `~/.m2` cache
- [] Student can state, in their own words, what each of the 6 lifecycle phases does

Discussion question for the group: Why does `mvn install` run `compile`, `test`, and `package` automatically? (Answer: Maven's lifecycle phases are sequential — running a later phase always runs every phase before it.)

2 Lab 2: Gradle Equivalent Build

Objective: Build the same project logic in Gradle, observe the task-based model, and directly compare it to Maven's lifecycle from Lab 1.

Prerequisites

- Gradle installed (`gradle -version`) or use the Gradle Wrapper

Steps

Step 1 — Initialize a new Gradle project

```
mkdir gradle-lifecycle-lab && cd gradle-lifecycle-lab
gradle init --type java-application --dsl groovy --test-framework junit
```

Step 2 — Inspect build.gradle

Open `app/build.gradle`. Compare its structure to the `pom.xml` from Lab 1 — note there's no fixed lifecycle listed; instead there are **tasks**.

Step 3 — List available tasks

```
./gradlew tasks
```

Identify the tasks that correspond to Maven phases: `compileJava`, `test`, `build`, `publish`.

Step 4 — Run the build and inspect incremental behavior

```
./gradlew build
./gradlew build
```

Run `build` a second time with no code changes. Observe that Gradle reports tasks as **UP-TO-DATE** instead of re-running them — this is the incremental build/cache behavior described in the source document.

Step 5 — Add the same dependency as Lab 1

```
dependencies {
    implementation 'com.fasterxml.jackson.core:jackson-databind:2.15.0'
}
```

```
./gradlew build --refresh-dependencies
```

Verification Checklist

- [] `./gradlew build` completes successfully
- [] Second run of `build` shows `UP-TO-DATE` tasks
- [] Student can map each Gradle task used to its Maven lifecycle equivalent from Lab 1

Comparison exercise: Time both builds with `time mvn clean install` vs `time ./gradlew clean build` on a clean checkout, then run each a second time without `clean`. Discuss why Gradle's second run is dramatically faster.

3 Lab 3: Transitive Dependency Conflicts

Objective: Deliberately create a version conflict through transitive dependencies, observe how Maven resolves it, and fix it using explicit version pinning.

Prerequisites

- Completed Lab 1 (uses the same `maven-lifecycle-lab` project)

Steps

Step 1 — Add two libraries that both depend on Jackson, at different versions

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.10.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>2.7.0</version>
</dependency>
```

Step 2 — Inspect the resolved dependency tree

```
mvn dependency:tree
```

Find every line mentioning `jackson-databind`. Two different versions will be pulled in transitively — identify which version Maven actually picked, and confirm it matches the "nearest wins" rule from the source document.

Step 3 — Force a specific version explicitly

Add an explicit top-level declaration to pin the version regardless of what any transitive dependency requests:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.15.0</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Step 4 — Re-run the dependency tree and confirm the fix

```
mvn dependency:tree
```

Confirm only version 2.15.0 now appears anywhere in the tree.

Verification Checklist

- [] Student can identify a version conflict from `mvn dependency:tree` output

- [] Student can explain "nearest wins" in their own words
- [] After adding `dependencyManagement`, only one Jackson version appears in the tree

Stretch goal: Repeat this same conflict in the Gradle project from Lab 2 using `./gradlew dependencies` and Gradle's `resolutionStrategy.force`. Compare the two approaches.

Lab Guide — companion to "Build Automation & Dependency Management" (Part 1 of 3).